M1S2 | OOP

Deep

Class is ...

an object’s draft which defines its attributes and methods

OOP Principles

Inheritance

allows you to create new classes based on existing classes, inheriting their attributes and methods

OOP Principles

Inheritance

allows you to create new classes based on existing classes, inheriting their attributes and methods

Polymorphism

allows objects of different classes to have the same methods, but with different implementations

Deep

OOP Principles

Inheritance

allows you to create new classes based on existing classes, inheriting their attributes and methods

Incapsulation

data and methods related to an object are combined in one place (class)

Polymorphism

allows objects of different classes to have the same methods, but with different implementations

Deep

OOP Principles

Inheritance

allows you to create new classes based on existing classes, inheriting their attributes and methods

Polymorphism

allows objects of different classes to have the same methods, but with different implementations

Incapsulation

data and methods related to an object are combined in one place (class)

Abstraction

allows you to highlight the general characteristics and functionality of objects or systems, ignoring the implementation details

OOP Pros & Cons

✅Modularity and Code Reuse

allows you to create classes that can serve as reusable modules

✅Improved Separation of Concerns

makes it easy to identify which part of the code is responsible for a specific functionality

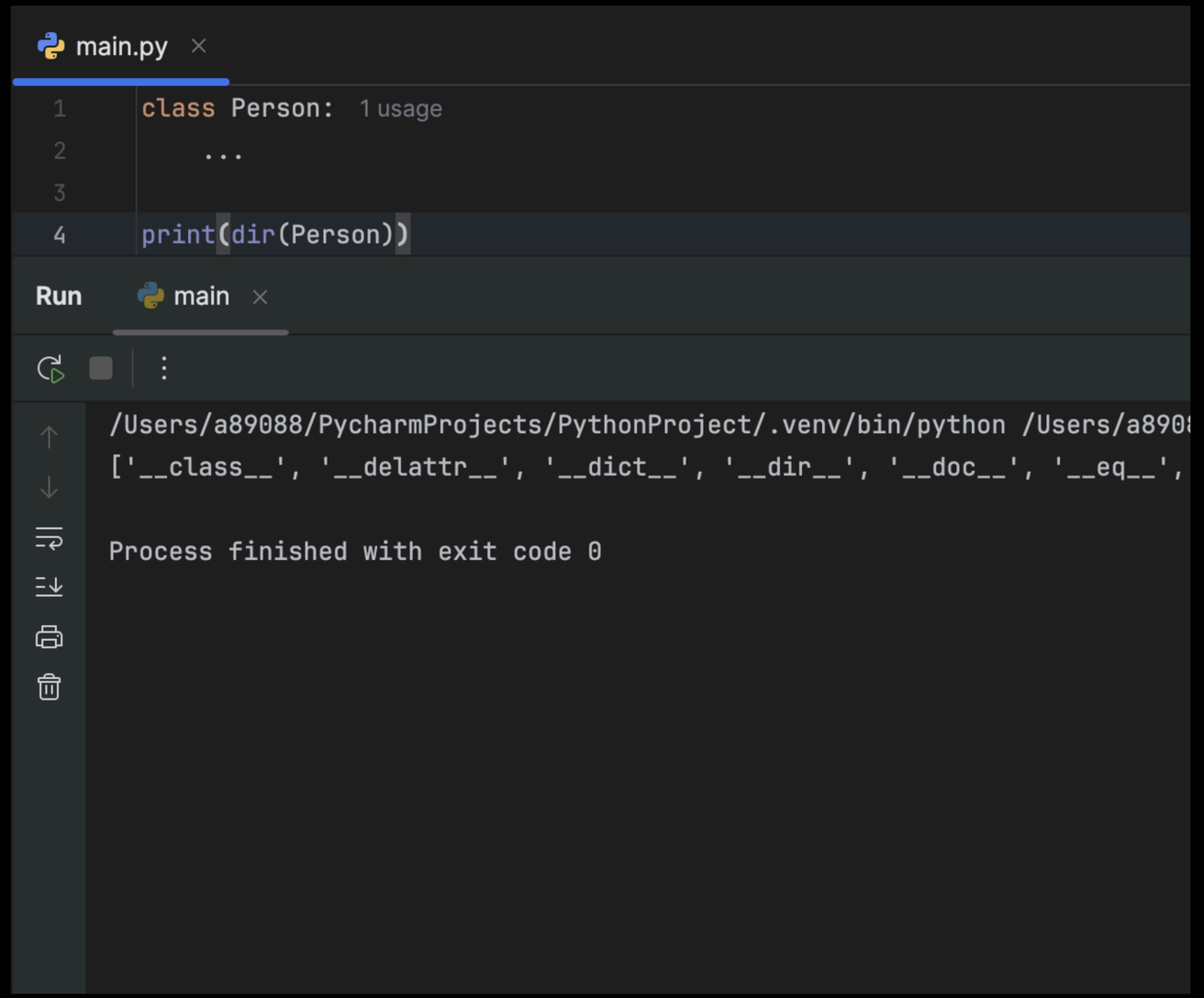
❌Tendency to Be Confusing

class hierarchies often lead to multiple method overridings

Deep

Lang Construction

Dunder(Magic) Methods

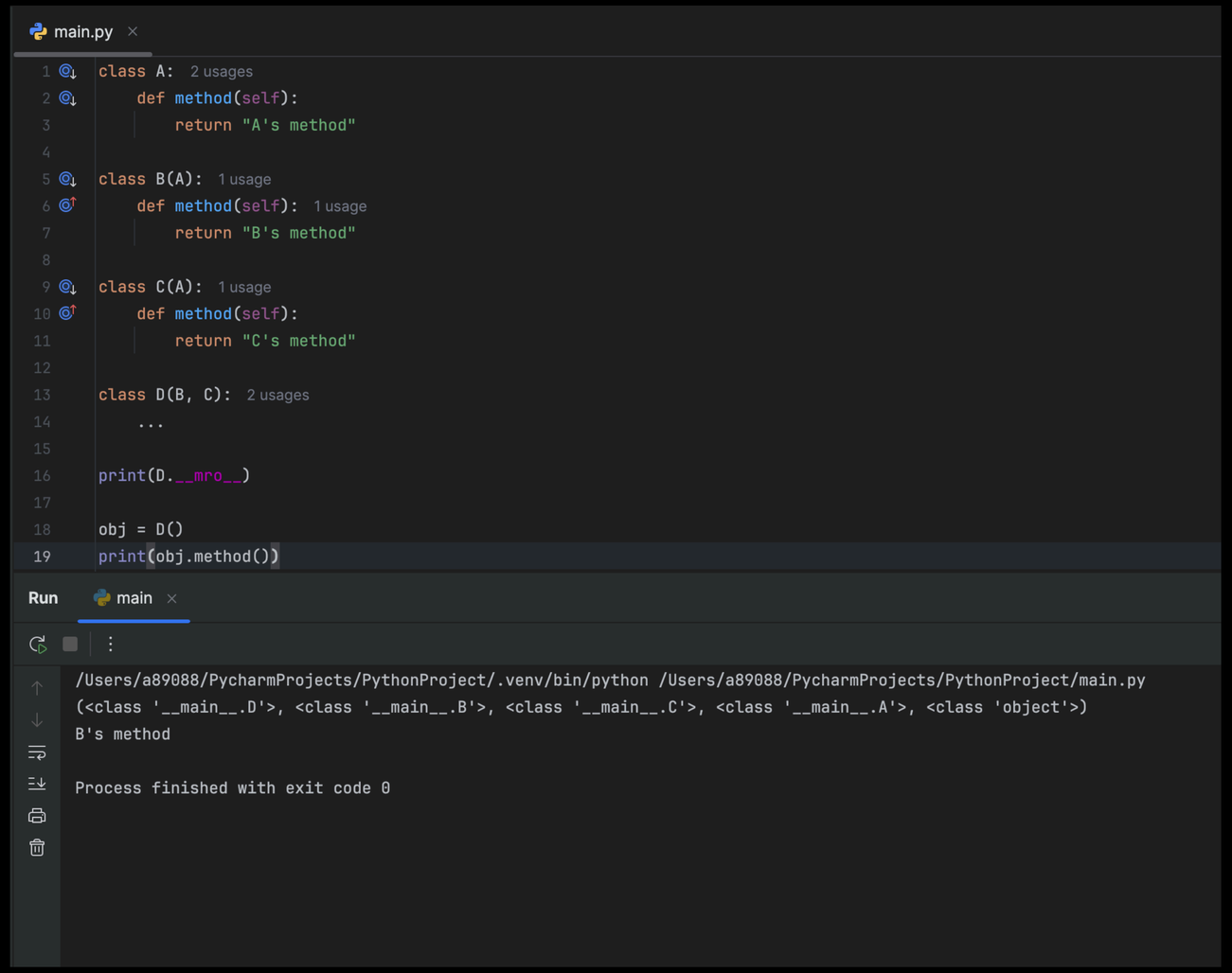


__init__, __str__, __repr__, __len__, __getitem__, __iter__, ...

Deep

MRO(Method Resolution Order)

Multiple Inheritance



algorithm for searching methods and attributes in inheritance hierarchy

Deep

SOLID💀

Single Responsibility

module must be responsible for one and only one actor

Open-Closed

software entities should be open for extension, but closed for modification

Liskov Substitution

functions that use a base type should be able to use subtypes of the base type without knowing about it

Interface Segregation

multiple client-specific interfaces are better than one general-purpose interface

Dependency Inversion

dependency on Abstractions. No dependency on implementation